

Codexa

Master Build Prompt for Antigravity

Purpose: Build a production-quality MVP of an AI-assisted code review, security auditing, and production-readiness platform for AI-generated codebases.

Based on: User-provided project synopsis, renamed from SecureReview AI to Codexa.

How to use this prompt

Copy the master prompt below into Antigravity as the project specification. Do not ask the tool to build every advanced feature at once. Instruct it to create the foundation first, then implement the milestones in order. Require it to keep the application runnable after every milestone, write tests, and explain any generated files or configuration changes.

The specification deliberately makes Java/Spring Boot the primary implementation and treats AI explanations as a controlled enhancement. Static analysis findings must remain deterministic and must never be replaced by an LLM guess.

Master prompt

BEGIN MASTER PROMPT

You are a senior full-stack engineer, application-security engineer, and software architect. Build a complete, runnable MVP named **Codexa**: an AI-assisted code review, security auditing, and production-readiness assessment platform for AI-generated or “vibe-coded” codebases.

1. Product vision

Codexa helps students, solo developers, hackathon teams, and early-stage startups answer: “Can this code safely move toward production?” Users submit a codebase as a ZIP file or a public GitHub repository URL. Codexa securely stages the code, performs deterministic static analysis and AST-based checks, detects security and maintainability risks, prioritizes findings, generates plain-English explanations and suggested fixes, and presents a transparent Production Readiness Score.

Codexa is an advisory tool, not a guarantee of security. The UI must clearly state that a clean scan does not prove the absence of vulnerabilities and that suggested fixes require human review and testing.

2. Non-negotiable constraints

- Never execute, compile, install, or run submitted user code. Treat all submitted repositories as untrusted text and configuration data.
- Primary backend: Java 17+ and Spring Boot 3.x. Use Maven unless the existing project convention requires Gradle.
- MVP analysis target: Java and Spring Boot projects. Design interfaces so JavaScript/TypeScript and Python can be added later, but do not let multi-language support delay the MVP.
- Use deterministic rules and JavaParser or Eclipse JDT for core findings. An LLM may explain and propose fixes but must not invent a finding that is presented as confirmed.
- Public GitHub repositories only for MVP. Do not implement private-repository OAuth unless the core product is complete.
- Keep secrets out of source control. Use environment variables and provide .env.example with placeholders.
- Use clear error messages, structured logging without secrets, unit tests, integration tests, and a concise README.

3. Required user experience

Create a polished but simple web application with these screens:

- Landing page: explain the problem, supported inputs, analysis stages, and disclaimer.
- New analysis page: tabs or cards for ZIP upload and public GitHub URL. Show limits before upload.
- Progress page: show job state such as queued, extracting, scanning, prioritizing, generating explanations, completed, or failed. Poll the backend or use Server-Sent Events if practical.
- Results dashboard: overall score, verdict, category scores, issue counts, top five actions, scan metadata, and a clear “not a security guarantee” disclaimer.

- Findings page/panel: filter by category, severity, confidence, file, and status. Each issue must show title, rule ID, OWASP mapping, file path, line range, evidence snippet, explanation, impact, remediation, suggested fix, confidence, and references.
- Code comparison view: show before and after snippets with copy buttons. Never silently overwrite user files.
- History page: list previous analyses if persistence is enabled. For the MVP, users may use an anonymous session; authentication is optional.

4. Supported ingestion

ZIP upload

- Endpoint: POST /api/v1/analyses/zip using multipart/form-data.
- Validate extension, content type where available, and configurable maximum compressed size, default 25 MB.
- Limit extracted size, file count, path depth, and individual file size. Reject archives that exceed limits or look like zip bombs.
- Prevent Zip Slip: normalize every entry path and verify that the resolved path remains inside the generated extraction directory before writing.
- Ignore binaries, images, build output, .git directories, node_modules, target, build, and other generated directories.
- Delete temporary files after analysis and use a scheduled cleanup as a fallback.

Public GitHub URL

- Endpoint: POST /api/v1/analyses/github with JSON {"repoUrl":"https://github.com/owner/repository"}.
- Accept only github.com HTTPS URLs for MVP. Reject malformed URLs, paths containing unsupported actions, and non-GitHub hosts.
- Parse owner and repository safely. Fetch metadata and an archive from the official GitHub API or documented archive endpoint, then send the downloaded archive through the exact same secure extraction pipeline as ZIP uploads.
- Implement timeouts, maximum download size, redirect validation, rate-limit friendly errors, and no arbitrary URL fetching. Never allow SSRF through user-controlled URLs.
- Store only the repository URL and commit/ref metadata needed for the report; do not claim a live scan if the repository changes later.

5. Analysis pipeline

```
[Input] -> [Validation] -> [Secure staging] -> [Project detection] -> [File filtering] -> [Java parsing]
-> [Security rules + quality rules] -> [Finding normalization] -> [Confidence/prioritization]
-> [Scoring] -> [Optional LLM explanation] -> [Persistence] -> [Dashboard/report]
```

Implement the pipeline behind an AnalysisOrchestrator service. Each stage should have a small interface so it can be tested independently. A failed optional AI explanation must not erase deterministic findings or mark the whole scan as failed.

6. Core security rules for MVP

Implement each rule with a stable rule ID, category, severity, confidence, explanation template, remediation template, OWASP mapping, and test fixtures containing vulnerable and safe examples.

Rule ID	Detection target	Initial behavior
CR-SQL-001	SQL injection	Flag SQL strings built through concatenation or interpolation with request-controlled values. Prefer AST/data-flow evidence; label regex-only results as lower confidence.
CR-CMD-001	Command injection	Flag Runtime.exec, ProcessBuilder, or shell commands receiving untrusted values.

Rule ID	Detection target	Initial behavior
CR-SEC-001	Hardcoded secrets	Detect likely passwords, tokens, API keys, private keys, and JWT secrets in source/config. Mask evidence in UI and never send full secrets to an LLM.
CR-AUTH-001	Missing access control	Identify sensitive controller endpoints and flag when no recognizable authorization annotation or service-level check is present. Mark as review-required rather than definitive when data flow is uncertain.
CR-XSS-001	Unsafe output	Flag direct rendering of untrusted input in HTML or response construction where escaping is absent.
CR-PASS-001	Weak password storage	Flag password persistence without BCrypt/SCrypt/Argon2-style hashing, or use of MD5/SHA-1 for passwords.
CR-LOG-001	Sensitive logging	Flag passwords, tokens, authorization headers, and high-risk PII being written to logs.
CR-CONFIG-001	Insecure configuration	Flag debug mode, permissive CORS, wildcard origins with credentials, disabled CSRF where unsafe, and exposed actuator endpoints.
CR-CRYPTO-001	Weak cryptography	Flag MD5/SHA-1 for security-sensitive hashing, hardcoded keys, ECB mode, and insecure random generation for security tokens.
CR-DEP-001	Dependency risk	Parse pom.xml/build.gradle and report dependency coordinates for which a configured advisory provider or local denylist has a known issue. If no provider is configured, report inventory only; do not fabricate CVEs.

Important: findings should distinguish confirmed pattern, likely pattern, and manual-review recommendation. Avoid claiming that the absence of @PreAuthorize automatically proves an authorization vulnerability.

7. Quality and production checks

Rule ID	Area	Detection
CR-QUAL-001	Method complexity	Calculate basic cyclomatic complexity from AST and flag configurable thresholds.
CR-QUAL-002	Long methods	Flag methods above a configurable line threshold, default 50.
CR-QUAL-003	Deep nesting	Flag nesting above a configurable threshold, default 4.
CR-QUAL-004	Duplication	Use normalized token or line shingles to find similar blocks; report pairs with a similarity threshold and suppress trivial getters/setters.
CR-QUAL-005	Architecture	Flag controllers that directly perform persistence or contain excessive business logic. Treat this as maintainability guidance, not a security defect.
CR-QUAL-006	Error handling	Flag empty catch blocks and broad catch(Exception) without meaningful handling.
CR-OPS-001	Configuration	Check for externalized secrets, environment-specific configuration, and production profile hygiene.
CR-OPS-002	Logging	Check for structured logging signals and ensure sensitive values are not logged.
CR-OPS-003	Testing	Detect whether test sources exist and provide a coverage hint; do not claim actual coverage without running tests.
CR-OPS-004	API readiness	Look for validation annotations, pagination hints on collection endpoints, health checks, and consistent error responses.

8. Finding model

```
Finding {
  String id; String ruleId; Category category; Severity severity; Confidence confidence;
  String title; String description; String impact; String remediation; String owaspMapping;
  String filePath; int startLine; int endLine; String evidenceMasked; String suggestedFix;
  double priorityScore; boolean requiresManualReview; List<String> references;
```

```
}
```

Use enums for category, severity, confidence, and job status. Every finding must include a deterministic source rule. Use a deduplication key based on rule ID, file path, line range, and normalized evidence.

9. Prioritization model

Build an explainable priority score first. A lightweight ML classifier may be added behind the same interface after the deterministic version works. Do not pretend that a heuristic is trained ML.

```
priority = severityWeight * confidenceWeight * exposureWeight * impactWeight
severity: CRITICAL=1.00, HIGH=0.80, MEDIUM=0.55, LOW=0.25
confidence: HIGH=1.00, MEDIUM=0.75, LOW=0.50
exposure: public endpoint=1.00, authenticated endpoint=0.75, internal/unknown=0.50
impact: auth/payment/PII/secrets=1.00, normal business data=0.70, low impact=0.40
```

Display why an issue received its priority. If you later add ML, train on synthetic and manually reviewed fixtures, expose model version and confidence, and retain the rule-based fallback.

10. Production Readiness Score

Calculate a transparent score from 0 to 100. Security must dominate the score, but quality and operations must be visible separately. Do not allow a large number of low-severity style findings to hide a critical security issue.

```
securityScore = max(0, 100 - securityPenalty)
qualityScore = max(0, 100 - qualityPenalty)
operationsScore = max(0, 100 - operationsPenalty)
overall = round(0.60 * securityScore + 0.25 * qualityScore + 0.15 * operationsScore)
if any confirmed CRITICAL security finding: verdict cannot exceed "Not ready"
if any HIGH finding in authentication, injection, secrets, or access control: verdict is at most "Needs urgent fixes"
```

Use configurable penalties, document them in the UI, and include a score breakdown. Suggested verdicts: 90–100 Review complete / low detected risk; 75–89 Generally promising / review findings; 50–74 Needs fixes before production; 0–49 Not ready. These labels are advisory, not certification.

11. LLM integration rules

- Make LLM integration optional and configurable through environment variables. The product must work without an LLM using deterministic templates.
- Send only the minimum necessary context: rule ID, masked evidence, nearby code, project language, and remediation guidance. Never send detected secrets, private keys, tokens, or full repositories.
- Use structured JSON output validated against a schema. If parsing fails, fall back to the deterministic explanation.
- The LLM must explain an existing finding, not create an unverified vulnerability. Add a prompt instruction: “Do not invent vulnerabilities, file paths, line numbers, CVEs, or dependencies.”
- Add caching by normalized rule/evidence hash to reduce cost and latency. Add timeout, retry limit, circuit breaker, and provider error handling.
- Label generated text as AI-assisted and require human review before applying fixes.

Use this internal explanation prompt template:

```
You are an application-security reviewer. Explain only the supplied finding.
Rule: {ruleId}
Category: {category}
Severity: {severity}
Confidence: {confidence}
File: {filePath}
Lines: {startLine}-{endLine}
Masked evidence: {evidence}
Known remediation: {remediation}
Return JSON only: {title, explanation, impact, remediation, suggestedFix, assumptions, references}.
Do not invent CVEs, line numbers, dependencies, or vulnerabilities. If context is insufficient, say
requires_manual_review=true.
```

12. Spring Boot API contract

Method	Endpoint	Purpose
POST	/api/v1/analyses/zip	Create analysis from multipart ZIP
POST	/api/v1/analyses/github	Create analysis from validated public GitHub URL
GET	/api/v1/analyses/{jobId}	Return status and complete results when available
GET	/api/v1/analyses/{jobId}/findings	Paginated/filterable findings
GET	/api/v1/analyses/{jobId}/report	Download or render a report
GET	/api/v1/health	Health check
GET	/api/v1/config/limits	Return public upload and scan limits

Use consistent error responses with timestamp, status, errorCode, message, and correlationId. Validate all request fields. Return 202 Accepted for asynchronous analysis jobs and 400/413/422/502/500 as appropriate.

13. Persistence model

Table	Important fields
analysis_job	id, source_type, source_identifier, repository_commit, status, progress_stage, created_at, completed_at, score, verdict, summary, error_code
finding	id, job_id, rule_id, category, severity, confidence, title, description, impact, remediation, file_path, start_line, end_line, evidence_masked, suggested_fix, priority_score, manual_review, references
analysis_metric	id, job_id, security_score, quality_score, operations_score, total_files, analyzed_files, duration_ms
llm_cache	cache_key, provider, model, response_json, created_at, expires_at

Use PostgreSQL for the complete version and an in-memory repository profile for local demo mode. Do not persist raw uploaded archives by default.

14. Package structure

```

com.codexa
analysis
controller
service
model
pipeline
scoring
ingestion
controller
github
zip
security
rules
api
security
quality
operations
ast
prioritization
ai
persistence
config
common
error
logging
validation

```

15. Secure implementation requirements

- Never call Runtime.exec, ProcessBuilder, javac, mvn, gradle, npm, pip, or any build command on submitted content.

- Use canonical-path checks during archive extraction and test malicious entries such as ../../outside.txt.
- Reject symlinks or handle them safely; do not allow extracted symlinks to escape the staging directory.
- Enforce compressed size, expanded size, file count, file depth, and per-file limits.
- Prevent SSRF: allow only GitHub HTTPS hostnames, resolve redirects carefully, use connect/read timeouts, and reject private IP destinations if any generic HTTP client is used.
- Mask secrets before persistence, display, logging, or LLM transmission.
- Apply API rate limiting, request size limits, CORS restrictions, secure headers, and centralized exception handling.
- Use path-safe file names and UUID-based staging directories. Delete staging data on completion/failure.
- Add a disclaimer that this is not a penetration test, runtime scanner, compliance certification, or guarantee of safety.

16. Testing requirements

- Unit tests for every rule with vulnerable, safe, and ambiguous examples.
- AST parser tests for line-number and snippet extraction.
- ZIP security tests for Zip Slip, zip bomb limits, too many files, oversized files, nested paths, symlinks, and malformed archives.
- GitHub ingestion tests for valid public URLs, invalid URLs, redirects, rate limits, unavailable repositories, and oversized downloads.
- End-to-end test: submit fixture -> job completes -> findings persist -> score returns -> dashboard renders.
- False-positive tests with secure parameterized SQL, BCrypt, environment variables, access annotations, and safe logging.
- LLM adapter contract tests using mocked responses, malformed JSON, timeouts, and provider failure.
- Security tests for Codexa itself: upload abuse, path traversal, SSRF, secret leakage, CORS, and error-message leakage.

17. Demo fixture projects

- codexa-demo-vulnerable: a small Spring Boot app containing intentionally vulnerable examples such as concatenated SQL, hardcoded secret, unsafe logging, weak hash, missing validation, and a long controller method.
- codexa-demo-secure: an equivalent app using parameterized queries, BCrypt, environment variables, validation, centralized errors, and service separation.
- codexa-demo-ambiguous: examples that should produce manual-review findings rather than definitive vulnerabilities.

Do not use real credentials or live exploit payloads. Mark all vulnerable fixtures clearly as educational and never deploy them publicly.

18. Delivery milestones

Milestone	Definition of done
M1: Foundation	Spring Boot app runs; frontend loads; health endpoint works; configuration and README exist.
M2: ZIP ingestion	Secure upload, limits, safe extraction, file filtering, cleanup, tests.
M3: Java analysis	JavaParser/JDT integration and normalized findings with line numbers.
M4: Rules	At least 8 security/quality/operations rules with fixtures and stable IDs.
M5: Score	Transparent score, category breakdown, verdict, top fixes.
M6: GitHub	Validated public URL ingestion through the same staging pipeline.
M7: AI layer	Optional structured explanations, masking, caching, fallback templates.

Milestone	Definition of done
M8: Dashboard	Progress, results, filters, code/fix view, history/report.
M9: Hardening	Security tests, rate limiting, cleanup, error handling, documentation.
M10: Demo	Secure and vulnerable fixtures, screenshots, architecture diagram, final report.

19. Output and documentation

- README with setup, environment variables, commands, architecture, limitations, and demo steps.
- OpenAPI/Swagger documentation for all API endpoints.
- Architecture diagram and data-flow diagram.
- Rule catalog documenting ID, category, detection logic, severity, confidence, false-positive risks, and remediation.
- Test report with detection results on vulnerable and secure fixtures.
- Sample JSON report and human-readable PDF/HTML report if practical.
- Security and privacy note explaining that submitted code is sensitive and how temporary data is handled.

20. Coding standards

- Use Java records where suitable for DTOs, Bean Validation, constructor injection, immutable configuration, and clear service boundaries.
- Use meaningful names, small methods, structured logging, and no sensitive values in logs.
- Avoid giant classes and TODO placeholders. If a feature cannot be implemented safely, create a clear interface and documented fallback.
- Prefer deterministic, testable code over clever abstractions.
- Keep frontend responsive and accessible. Show loading, empty, error, and partial-results states.
- At the end of each implementation milestone, run tests and report exactly what works, what is incomplete, and how to run it.

21. Final acceptance criteria

- A user can upload a ZIP and receive a completed analysis without submitted code being executed.
- A user can submit a valid public GitHub repository URL and receive the same analysis pipeline results.
- The system detects the agreed security and quality fixtures with file paths and line numbers.
- The system avoids or clearly labels ambiguous findings and does not fabricate CVEs or dependency vulnerabilities.
- The dashboard displays score, verdict, category breakdown, filters, findings, and remediation.
- The system continues to work if the LLM is disabled or unavailable.
- Archive extraction, SSRF, upload limits, masking, cleanup, and error handling are tested.
- The project can be run locally from the README with documented environment variables.

Implementation instruction: Start by generating the project plan and repository structure, then implement M1 and M2. Do not jump directly to the LLM. After each milestone, run tests, show the changed files, and wait for approval before beginning the next milestone. **END MASTER PROMPT**

Recommended Antigravity execution order

- Paste the master prompt and ask Antigravity to produce a plan plus directory tree only.
- Approve foundation and ZIP ingestion first.

- Add deterministic Java rules and test fixtures.
- Add scoring and dashboard.
- Add public GitHub ingestion.
- Add optional LLM explanations only after masking and fallback behavior are complete.
- Finish with security testing, documentation, and a demo using intentionally vulnerable local fixtures.

Important product decisions

Decision	Recommendation
Primary name	Codexa
MVP language	Java/Spring Boot only
Inputs	ZIP and public GitHub URL
Analysis	Deterministic SAST/AST first; AI explanation second
ML	Explainable prioritization first; real classifier later
Persistence	PostgreSQL profile plus in-memory demo profile
LLM	Optional, schema-validated, masked context, fallback templates
Safety	Never execute or compile user code
Product claim	Advisory production-readiness assessment, not a security certification

Sources used for the specification

The project synopsis you uploaded was used as the primary source for the scope, objectives, architecture, technology choices, rule categories, timeline, testing strategy, and expected outcomes. The OWASP Top 10:2025 provides the security-risk framing, and GitHub documentation supports the public repository/archive ingestion approach.

[file:98][web:1][web:2]